

APP_USER			
string	Id	PK	Identity UserId
string	Email	UK	
string	FullName		

ADDRESS		
int	AddressId	PK
string	UserId	FK
string	Country	
string	City	
string	Street	
string	Zip	
bool	IsDefault	

CATEGORY			
int	CategoryId	PK	
string	Name	UK	
int	ParentCategoryId	FK	nullable

PRODUCT		
int	ProductId	PK
int	CategoryId	FK
string	Name	
string	SKU	UK
decimal	Price	
int	StockQuantity	
bool	IsActive	
datetime	CreatedAt	

ORDER		
int	OrderId	PK
string	UserId	FK
int	ShippingAddressId	FK
string	OrderNumber	UK
int	Status	
datetime	OrderDate	
decimal	TotalAmount	

ORDER_ITEM		
int	OrderItemId	PK
int	OrderId	FK
int	ProductId	FK
decimal	UnitPrice	
int	Quantity	
decimal	LineTotal	

Conceptual relationships

- Category 1—∞ Product
- Customer (Identity User) 1—∞ Order
- Order 1—∞ OrderItem
- Product 1—∞ OrderItem
- Customer 1—∞ Address
- Order ∞—1 Address (shipping)

MVC Project Type and Structure

Project template

- ASP.NET Core Web App (Model-View-Controller)
- Authentication: Individual Accounts (ASP.NET Core Identity)
- Database: SQL Server
- EF Core with migrations - Code First

MVC Pages (Views) You Should Build

Public

- **Catalog/Index**: browse products + filter by category + search + pagination
- **Catalog/Details/{id}**
- **Cart/Index**
- **Cart/Add/{productId}**, **Cart/Update**, **Cart/Remove**
- **Checkout/Index** (address + confirm)
- **Orders/Index** (my orders)
- **Orders/Details/{id}**

Admin

- **Admin/Categories** CRUD
- **Admin/Products** CRUD
- **Admin/Orders** list + update status

Controllers and Responsibilities

CatalogController

- `Index(categoryId, q, sort, page)`
- `Details(id)`

CartController

- `Index()`
- `Add(productId)`
- `Update(itemId, qty)`
- `Remove(itemId)`
- Cart storage choice:
 - **Session-based cart** (simple)
 - Or DB cart tables (more advanced)

OrdersController

- Checkout () GET
- Checkout (CheckoutVM) POST
- Index () (customer orders)
- Details (id)

Admin Area Controllers

- Admin authorization: `[Authorize(Roles = "Admin")]`
- Products CRUD + category selection
- Orders status updates

ViewModels (Important in MVC)

Use ViewModels to avoid passing EF entities directly to views.

Examples:

- ProductListVM (items + paging + filter state)
- ProductDetailsVM
- CartVM (items + totals)
- CheckoutVM (selected address + cart summary)
- OrderDetailsVM (order header + items)
-

Checkout transaction (must be atomic)

During checkout POST:

1. Validate stock for all items
2. Create Order
3. Create OrderItems
4. Decrease product stock
5. Save changes inside one transaction

Notes

- Start with CRUD operations for all entities.
- Use view models.
- Use repositories.
- We will add authentication and authorization after authentication lectures.